

Sleep and Wakeup

Blocking instead of wasting CPU time

Definition

Sleep is a system call that blocks the calling process. **Wakeup** is a system call that makes a blocked process ready to run again.

1. Why Sleep and Wakeup Are Needed

Main Idea

Peterson's solution, TSL, and XCHG can correctly provide mutual exclusion, but they require busy waiting. A waiting process keeps checking a condition in a tight loop instead of giving up the CPU.

Busy-Waiting Cost

Busy waiting wastes CPU time because the process is running but doing no useful work. It can also create scheduling problems when priorities are involved.

Blocking Alternative

Instead of spinning, a process that cannot continue can go to sleep. Another process wakes it up when the condition it needs becomes true.

2. Priority Inversion Problem

Priority Inversion

Priority inversion can occur when a high-priority process waits for a resource held by a low-priority process, but the low-priority process cannot run long enough to release it.

Consider two processes:

- **H**: high-priority process.
- **L**: low-priority process.

How the Problem Happens

Suppose L is inside its critical region. Then H becomes ready to run and starts busy waiting for the same critical region. Because H has higher priority, the scheduler keeps running H. But H is only spinning, and L never gets CPU time to leave the critical region.

Result

H waits forever even though the only process that can help it is L. The high-priority process prevents the low-priority process from releasing the resource.

Step	Event
1	Low-priority process L enters the critical region.
2	High-priority process H becomes ready to run.
3	Scheduler runs H because it has higher priority.
4	H busy-waits for the critical region.
5	L cannot run, so it cannot leave the critical region.
6	H keeps spinning forever.

3. Sleep and Wakeup Primitives

Sleep

`sleep()` blocks the caller. The process is suspended and does not consume CPU time while waiting.

Wakeup

`wakeup(process)` wakes a specified sleeping process. Some systems instead use `sleep(address)` and `wakeup(address)`, where the address identifies the condition being waited for.



4. Producer-Consumer Problem

Bounded-Buffer Problem

The **producer-consumer problem**, also called the **bounded-buffer problem**, has two processes sharing a fixed-size buffer. The producer inserts items, and the consumer removes items.

Process	Role
Producer	Creates items and puts them into the shared buffer.
Consumer	Removes items from the shared buffer and uses them.
Buffer	Holds at most N items.
count	Shared variable storing the current number of items in the buffer.

Correct Behavior Wanted

If the buffer is full, the producer should sleep until the consumer removes an item. If the buffer is empty, the consumer should sleep until the producer inserts an item.

5. Naive Sleep-Wakeup Code

```
#define N 100 /* number of slots in the buffer */
int count = 0; /* number of items in the buffer */

void producer(void)
{
    int item;

    while (TRUE) {
        item = produce_item();
        if (count == N)
            sleep();
        insert_item(item);
        count = count + 1;
        if (count == 1)
            wakeup(consumer);
    }
}

void consumer(void)
{
    int item;

    while (TRUE) {
        if (count == 0)
            sleep();
        item = remove_item();
        count = count - 1;
        if (count == N - 1)
            wakeup(producer);
        consume_item(item);
    }
}
```

Fatal Weakness

The shared variable `count` is accessed without protection. The test of `count` and the call to `sleep()` are not one atomic operation.

6. The Lost-Wakeup Race

Initial State

Assume the buffer is empty, so `count = 0`. The consumer is about to check whether it should sleep.

Step	Consumer	Producer
1	Reads <code>count</code> and sees 0.	–
2	Is interrupted before actually calling <code>sleep()</code> .	–
3	–	Runs and inserts an item.
4	–	Increments <code>count</code> to 1.
5	–	Calls <code>wakeup(consumer)</code> because the buffer was empty.
6	Wakeup is lost because the consumer is not yet sleeping.	–
7	Runs again and continues using the old decision that <code>count</code> was 0.	–
8	Calls <code>sleep()</code> even though an item is available.	–

Why This Is Serious

Eventually the producer may fill the buffer and go to sleep too. Then the consumer is asleep, the producer is asleep, and no process is left to wake the other.

7. Core Cause of the Bug

Lost Wakeup

A **lost wakeup** occurs when a wakeup is sent to a process that is not yet sleeping. Since the signal is not saved, the process may later go to sleep forever.

The Exact Race

The consumer checks `count == 0`, but before it can sleep, the producer changes `count` and sends a wakeup. The consumer misses the wakeup and then sleeps based on stale information.

Pattern to Notice

The danger is not only shared data. The danger is checking a condition and blocking on that condition without making the two actions atomic.

8. Wakeup Waiting Bit

Quick Fix Idea

One proposed patch is a **wakeup waiting bit**. If a wakeup is sent before the target process is asleep, the bit is set. Later, when the process tries to sleep, it sees the bit, clears it, and stays awake.

Situation	Behavior with Wakeup Waiting Bit
Wakeup arrives while process sleeps	Process is woken normally.
Wakeup arrives before process sleeps	Wakeup bit is saved.
Process later tries to sleep	Saved bit is cleared, and the process remains awake.

Why This Is Not Enough

One wakeup waiting bit may work for this simple two-process example, but it does not solve the general problem. With three or more processes, one bit is not enough to remember all possible missed wakeups.

9. Busy Waiting vs. Blocking

Method	Behavior	Main Problem
Busy waiting	Process repeatedly checks until it can proceed.	Wastes CPU time and can cause priority inversion.
Sleep/wakeup	Process blocks and waits to be awakened.	Can lose wakeups if condition checks and blocking are not protected.

10. Quick Review

Key Points

- Busy waiting wastes CPU time because the process spins in a loop.
- Priority inversion can occur when a high-priority process spins while a low-priority process holds the needed resource.
- `sleep()` blocks a process so it does not consume CPU time.
- `wakeup()` makes a sleeping process ready again.
- The producer-consumer problem uses a fixed-size shared buffer.
- The producer sleeps when the buffer is full.
- The consumer sleeps when the buffer is empty.
- The naive solution fails because a wakeup can be lost.
- A wakeup waiting bit is only a patch, not a general solution.